

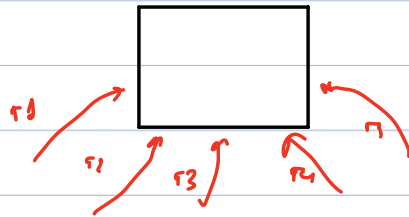
Good evening everyone



class start at 8:10

Q) How many threads are allowed in critical section at one time.

1



race condition
wrong output

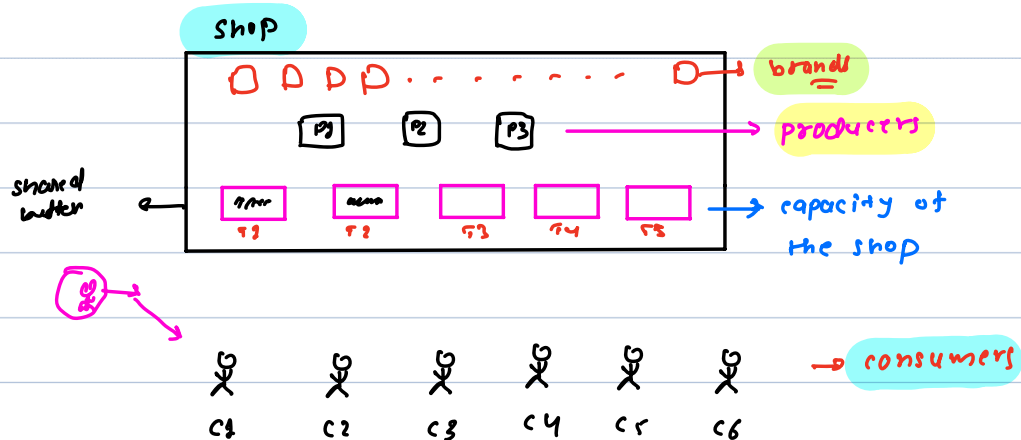
mutex → implement locks

Semaphores

dijkstra

1965

Producer - consumer problem



- maximum number of cloths producer can present is equal to total number of table.

- producer can represent only on empty tables.



producer can represent 3 more cloths.



producer wait until the table is not empty.



- if the all the table is empty the customer cannot purchase.



two customer can buy the cloths.

we have to make a program for this problem.

maxLimit

Queue

producer

add into queue



consumer

remove from the queue

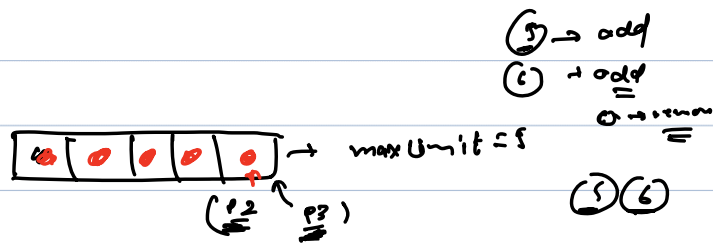


producer

```
if (q.size() < maxLimit) {  
    q.add(new brand());  
}
```

consumer

```
if (q.size() > 0) {  
    q.remove();  
}
```



producer 2

```
if (q.size() < maxLimit) {  
    q.add(new brand());  
}
```

producer 3

```
if (q.size() < maxLimit) {  
    q.add(new brand());  
}
```

lets try **mutex mechanism**

lock the entire queue.

this method is slow.

if multiple thread are try to lock.



dead lock arises in this situation.



Semaphores

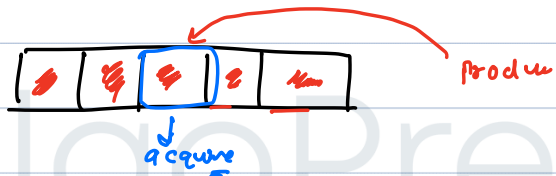
it is also control the access to shared resources.

```
Semaphore s = new Semaphore(n);
```

↓
Permit

$n \rightarrow$ how many resources you can access.

```
s.acquire(); // LOCK
```



Initially we have total n available resource (empty table);

when we acquire then n is decrease by 1

```
s.release();
```

when we release then n is increased. by 1

```
break fill g:20
```

Initially all table are empty.

```
Semaphore prodsem = new Semaphore(5);
```

↓
available res

```
Semaphore consema = new Semaphore(0);
```

prod
semu 5 4 3 4



cons
sema 0 1 2 1

Producer

```
prodsem.acquire();  
q.add(new cloth());  
conssem.release();
```

consumer

```
conssem.acquire();  
q.remove();  
prodsem.release();
```

Semaphore s : new Semaphore(1); → mutex
lock

it can handle multiple resources

when you acquire it send notification or signal.

it is also check that resource is available or not.

prodsem.acquire()

q.add(new cloth());

conssem.release();

if (number == 0) {

throw InterruptedException;

}

when we use acquire then we have to handle the exception.

try {

prodsem.acquire()

} catch (Exception e) {

sys.o(r);

3



```
1 import java.util.Queue;
2 import java.util.concurrent.ConcurrentLinkedDeque;
3 import java.util.concurrent.Semaphore;
4
5 class Producer implements Runnable{
6     private Queue<Cloths> q;
7     private int maxLimit;
8     private String name;
9     private Semaphore prodSema;
10    private Semaphore consSema;
11
12    public Producer(Queue<Cloths> que , int mL , String n , Semaphore ps , Semaphore cs){
13        this.q = que;
14        this.maxLimit = mL;
15        this.name = n;
16        this.prodSema = ps;
17        this.consSema = cs;
18    }
19
20    public void run(){
21        int i = 100000000;
22        while(i-->0){
23            try {
24                prodSema.acquire();
25                System.out.println(q.size() + " add to Queue "+ " Prod name " +name);
26                q.add(new Cloths());
27                consSema.release();
28            } catch (InterruptedException e) {
29
30                e.printStackTrace();
31            }
32
33        }
34    }
35 }
36 }
37
```

```

38 class Consumer implements Runnable{
39     private Queue<Cloths> q;
40     private String name;
41     private Semaphore prodSema;
42     private Semaphore consSema;
43
44     public Consumer(Queue<Cloths> que , String n, Semaphore ps , Semaphore cs){
45         this.q = que;
46         this.name = n;
47         this.prodSema = ps;
48         this.consSema = cs;
49
50     public void run(){
51         int i = 100000000;
52         while(i > 0){
53             Semaphore consSema
54             consSema.acquire();
55             System.out.println(q.size() + " Remove from Queue "+ " Cons name " +name);
56             q.remove();
57             prodSema.release();
58         } catch (InterruptedException e) {
59             e.printStackTrace();
60         }
61
62     }
63
64 }
65
66 class Cloths{
67     String colr;
68     String brand;
69 }
70 public class task5 {
71     Run | Debug
72     public static void main(String[] args) {
73         int maxLimit = 5;
74         Queue<Cloths> q = new ConcurrentLinkedDeque<>();
75
76         Semaphore prodSema = new Semaphore(maxLimit);
77         Semaphore consSema = new Semaphore(permits:0);
78
79         Producer p1 = new Producer(q ,maxLimit , n:"Prod1",prodSema ,consSema);
80         Producer p2 = new Producer(q ,maxLimit , n:"Prod2",prodSema ,consSema);
81         Producer p3 = new Producer(q ,maxLimit , n:"Prod3",prodSema ,consSema);
82     }

```

```
81
82     Consumer c1 = new Consumer(q , n:"Cons1",prodSema ,consSema);
83     Consumer c2 = new Consumer(q , n:"Cons2",prodSema ,consSema);
84     Consumer c3 = new Consumer(q , n:"Cons3",prodSema ,consSema);
85     Consumer c4 = new Consumer(q , n:"Cons4",prodSema ,consSema);
86     Consumer c5 = new Consumer(q , n:"Cons5",prodSema ,consSema);
87     // Consumer c6 = new Consumer(q , "Cons6");
88     // Consumer c7 = new Consumer(q , "Cons7");
89
90     Thread tp1 = new Thread(p1);
91     tp1.start();
92     Thread tc1 = new Thread(c1);
93     tc1.start();
94     Thread tc2 = new Thread(c2);
95     tc2.start();
96
97     Thread tp2 = new Thread(p2);
98     tp2.start();
99     Thread tc3 = new Thread(c3);
100    tc3.start();
101    Thread tc4 = new Thread(c4);
102    tc4.start();
103
104    Thread tp3 = new Thread(p3);
105    tp3.start();
106    Thread tc5 = new Thread(c5);
107    tc5.start();
108    // Thread tc6 = new Thread(c6);
109    // tc6.start();
110    // Thread tc7 = new Thread(c7);
111    // tc7.start();
112
113 }
114 }
```
